

INTRODUCTION

In today's digital era, banking systems handle millions of transactions every day, making them highly vulnerable to fraudulent activities. Detecting such fraud using traditional methods is often difficult, as these approaches focus mainly on individual transactions rather than the relationships between accounts. To overcome this limitation, this project applies concepts from Graph Theory and Network Analysis to identify suspicious patterns in banking transactions.

In this project, bank accounts are represented as nodes and transactions between them are represented as edges, forming a network structure. By analyzing this network, hidden connections and unusual transaction behaviors can be identified more effectively. Centrality measures such as degree centrality and betweenness centrality are used to determine the importance and influence of each account within the network.

Accounts with unusually high connectivity or those acting as intermediaries in multiple transactions are flagged as potentially suspicious. This approach helps in detecting fraud rings, money laundering loops, and abnormal transaction flows. Overall, the project demonstrates how data analysis and network-based techniques can enhance fraud detection and improve the security of banking systems.

PROBLEM STATEMENT

In modern banking systems, a large number of financial transactions are carried out daily, making it challenging to monitor and detect fraudulent activities efficiently. Traditional fraud detection methods mainly focus on analyzing individual transactions based on predefined rules, such as transaction limits or unusual amounts. However, these methods often fail to identify complex fraud patterns that involve multiple accounts working together.

The main problem addressed in this project is to detect suspicious financial activities by analyzing the relationships between bank accounts rather than just individual transactions. Fraudsters often operate in networks, performing a series of interconnected transactions to hide illegal activities such as money laundering

or unauthorized fund transfers. Identifying such patterns requires a deeper analysis of how accounts interact with each other.

To solve this problem, the project uses concepts from **Graph Theory** and **Network Analysis**. The goal is to construct a transaction network where accounts are nodes and transactions are edges, and then analyze this network using centrality measures to identify unusual patterns. By doing so, the system can highlight suspicious accounts and detect potential fraud networks more effectively than traditional approaches.

THEORETICAL BACKGROUND

The theoretical background refers to the fundamental concepts and principles used in the development of this project. The fraud detection system in banking transactions is based on concepts from Graph Theory, Network Analysis, and data analysis techniques. Understanding these concepts helps in identifying suspicious transaction patterns and detecting fraudulent activities.

1. Banking Transactions

Banking transactions involve the transfer of money between accounts. These transactions occur frequently and include deposits, withdrawals, and fund transfers. In large banking systems, thousands of transactions take place daily, making it difficult to manually monitor and detect fraudulent activities.

2. Graph Representation

Graph representation is used to model the transaction network. In this approach:

- Each bank account is represented as a **node**
- Each transaction is represented as an **edge**

The graph is usually directed, where the direction indicates the flow of money from sender to receiver. This representation helps in visualizing relationships between accounts.

3. Link Analysis

Link analysis is a technique used to evaluate relationships between entities in a network. In fraud detection, it helps in identifying hidden connections between accounts and detecting suspicious transaction patterns such as loops and clusters.

4. Centrality Measures

Centrality measures are used to identify important nodes in a network:

- **Degree Centrality:** Measures the number of connections an account has. High values indicate highly active accounts.
- **Betweenness Centrality:** Identifies accounts that act as intermediaries between other accounts.
- **Closeness Centrality:** Measures how quickly an account can interact with others in the network.

These measures help in detecting suspicious accounts involved in fraud.

5. Fraud Patterns

Fraudulent activities often show specific patterns such as:

- Circular transactions (money loops)
- Highly connected accounts (hub nodes)
- Frequent high-value transactions
- Isolated clusters of accounts

Identifying these patterns helps in detecting fraud networks.

6. Data Visualization

Data visualization is used to represent transaction data in graphical form. Network graphs help in clearly showing relationships between accounts and highlighting suspicious nodes. Visualization makes it easier to understand complex transaction data.

7. Python Libraries

The project uses Python libraries such as:

- Pandas for data handling
- NetworkX for graph analysis
- Matplotlib for visualization

These tools help in building and analyzing the fraud detection system efficiently.

ALGORITHM EXPLANATION

Step 1: Data Collection

The transaction dataset is collected, which contains details such as sender account, receiver account, transaction amount, and date. This data forms the basis for analysis.

Step 2: Data Loading

The collected dataset is loaded into the system using Python. The data is read using appropriate libraries and stored in a structured format for further processing.

Step 3: Data Preprocessing

The dataset is checked for missing or incorrect values. Column names are cleaned, and necessary fields such as sender, receiver, and amount are selected for analysis.

Step 4: Graph Construction

A graph is created using concepts from **Graph Theory**.

- Each bank account is represented as a node
- Each transaction is represented as a directed edge
- The transaction amount is assigned as edge weight

Step 5: Network Formation

All nodes and edges are connected to form a transaction network. This network represents the flow of money between accounts and helps in identifying relationships.

Step 6: Centrality Calculation

Centrality measures are calculated using **Network Analysis** techniques:

- Degree Centrality
- Betweenness Centrality

These measures help in identifying important and highly connected accounts.

Step 7: Fraud Detection

Accounts are analyzed based on centrality values. Accounts with high connectivity or those acting as intermediaries are marked as suspicious. Patterns such as loops and clusters are also considered.

Step 8: Data Visualization

The transaction network is visualized using graphs. Suspicious accounts are highlighted using different colors, making it easier to identify fraud patterns.

Step 9: Result Display

The system displays the detected suspicious accounts along with graphical output. This helps users understand the analysis clearly and interpret the results effectively.

IMPLEMENTATION

fraud.py

```
import pandas as pd
```

```
import networkx as nx
```

```

import matplotlib.pyplot as plt

data = pd.read_csv("transactions.csv")
data.columns = data.columns.str.strip()

G = nx.from_pandas_edgelist(
    data,
    source='Sender',
    target='Receiver',
    edge_attr='Amount',
    create_using=nx.DiGraph()
)

degree = nx.degree_centrality(G)
betweenness = nx.betweenness_centrality(G)
suspicious = []

for node in G.nodes():
    if degree[node] > 0.3 or betweenness[node] > 0.05:
        suspicious.append(node)

print("\n🔍 FRAUD ANALYSIS REPORT")
print("-" * 30)

print("\n📌 High Degree Accounts:")

for node, val in degree.items():
    if val > 0.3:
        print(f"{node} → {round(val,3)}")

print("\n📌 High Betweenness Accounts:")

```

```

for node, val in betweenness.items():
    if val > 0.05:
        print(f"{node} → {round(val,3)}")
print("\n🚨 Suspicious Accounts Detected:")
for node in suspicious:
    print(node)
plt.figure(figsize=(10,7))
pos = nx.spring_layout(G)
node_colors = []
for node in G.nodes():
    if node in suspicious:
        node_colors.append('red')
    else:
        node_colors.append('lightblue')
nx.draw(G, pos, with_labels=True, node_color=node_colors, node_size=2000)
edge_labels = nx.get_edge_attributes(G, 'Amount')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
plt.title("Fraud Transaction Network Analysis")
plt.show()

```

DATASET USED

transactions.csv

Sender,Receiver,Amount,Date

A101,B205,5000,1/1/2024

A101,C309,20000,1/2/2024

B205,C309,7000,1/3/2024

C309,D410,15000,1/4/2024

D410,E511,30000,1/5/2024

E511,A101,12000,1/6/2024

B205,F612,8000,1/7/2024

F612,G713,22000,1/8/2024

G713,A101,50000,1/9/2024

A101,H814,11000,1/10/2024

H814,I915,9000,1/11/2024

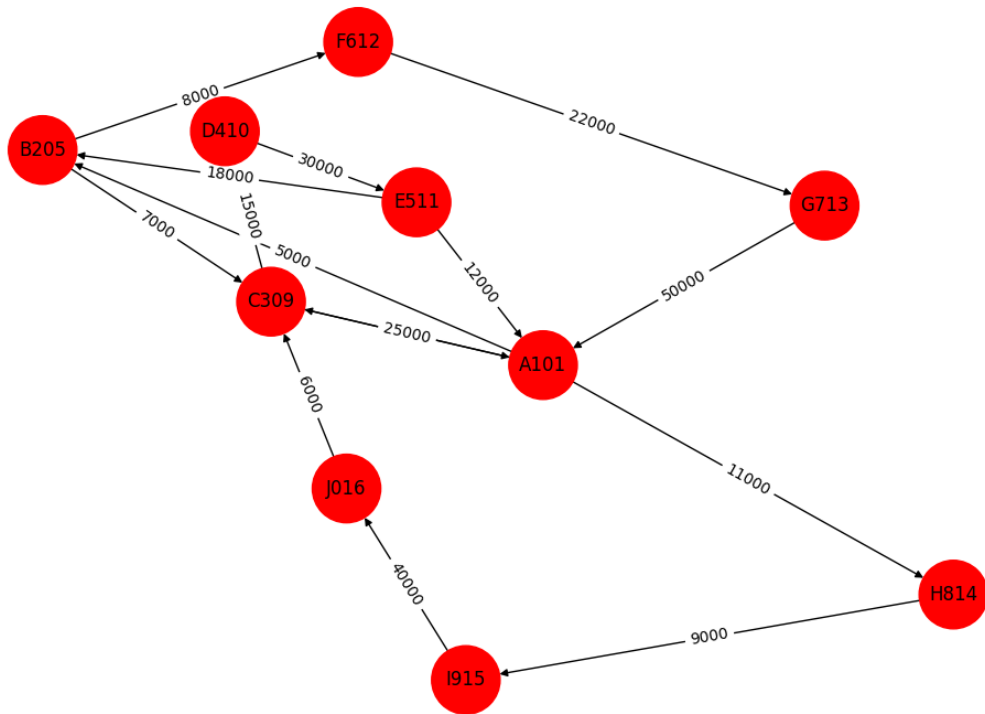
I915,J016,40000,1/12/2024

J016,C309,6000,1/13/2024

C309,A101,25000,1/14/2024

E511,B205,18000,1/15/2024

OUTPUT



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\visual studio code\daminiproject> & 'c:\Users\visual studio code\daminiproject\.venv\Scripts\
\bundled\libs\debugpy\launcher' '54139' '--' 'C:\Users\visual studio code\daminiproject\fraud.py'
🔥 High Betweenness Accounts:
A101 → 0.604
B205 → 0.229
C309 → 0.472
D410 → 0.111
E511 → 0.111
F612 → 0.111
G713 → 0.111
H814 → 0.208
I915 → 0.208
J016 → 0.208

🚩 Suspicious Accounts Detected:
A101
B205
C309
D410
E511
F612
G713
H814
I915
J016
█
```

RESULTS AND FUTURE ENHANCEMENT

Results:

The developed system successfully analyzes banking transaction data and identifies suspicious accounts using link analysis techniques. By applying concepts from **Graph Theory** and **Network Analysis**, the project converts transaction data into a network structure for better analysis.

The graphical representation of the transaction network helps in visualizing the relationships between accounts. Accounts with high degree centrality and betweenness centrality are identified as potentially suspicious. These accounts are highlighted in the network graph, making it easier to detect unusual transaction behavior.

The system is able to:

- Identify highly connected accounts involved in multiple transactions
- Detect intermediary accounts acting as bridges between others
- Recognize transaction loops and clustered patterns
- Provide a clear visualization of fraud-prone areas

Overall, the project demonstrates how data analysis and network-based techniques can effectively detect potential fraud in banking transactions.

Future Enhancements:

The project can be further improved by incorporating advanced features and technologies:

- Integration with real-time banking transaction data for live fraud detection
- Implementation of machine learning algorithms for more accurate prediction of fraudulent activities
- Addition of fraud scoring system to rank accounts based on risk level
- Use of advanced network analysis techniques such as community detection

- Development of an interactive dashboard for better visualization and user experience
- Handling large-scale datasets efficiently using big data technologies

These enhancements can make the system more powerful, accurate, and suitable for real-world banking applications.

GITHUB LINK

<https://github.com/NarkeeshJ/Fraud-Detection-Banking>

REFERENCES

1. Python Software Foundation. *Python Documentation*.
Available at: <https://docs.python.org/>
2. Wes McKinney (2010). *Data Structures for Statistical Computing in Python*.
Proceedings of the 9th Python in Science Conference.
3. Pandas Development Team. *Pandas Documentation*.
Available at: <https://pandas.pydata.org/docs/>
4. Matplotlib Development Team. *Matplotlib Documentation*.
Available at: <https://matplotlib.org/stable/contents.html>
5. Aric Hagberg, Daniel Schult, Pieter Swart (2008).
Exploring Network Structure, Dynamics, and Function using NetworkX.
6. Jiawei Han, Micheline Kamber, Jian Pei (2011).
Data Mining: Concepts and Techniques. Morgan Kaufmann Publishers.
7. Linton C. Freeman (1978).
Centrality in Social Networks: Conceptual Clarification.
Social Networks Journal.
8. Kaggle. *Financial Transaction Datasets*.
Available at: <https://www.kaggle.com/>